Antes de codar: como pensar um software

Do problema à solução: requisitos, decisões, arquitetura, planejamento e validação.

Pergunta central: como saber o que construir — e por quê?

O erro de começar pelo código

- “Vou usar Java, React e PostgreSQL.” Mas qual problema resolvemos?
- Código é implementação; desenvolvimento começa com entendimento.
- Uma decisão inicial ruim pode orientar meses de retrabalho.

O caminho do problema até a entrega

01 · Descobrir

Contexto, usuários e dor real

02 · Definir

Escopo e requisitos

03 · Projetar

Fluxos, dados e arquitetura

04 · Planejar

Prioridades, tarefas e riscos

05 · Construir

Código, testes e integração

06 · Aprender

Medição, feedback e evolução

1. Entenda o problema

- Quem é afetado? O que tenta fazer?
- Qual situação atual gera perda de tempo, erro ou custo?
- Como essa pessoa resolve isso hoje?
- Qual resultado concreto demonstraria sucesso?

2. Transforme a dor em requisitos

Funcionais

O que o sistema deve fazer: cadastrar, pedir, pagar

Não funcionais

Como deve funcionar: segurança, desempenho, disponibilidade

Restrições

Prazo, orçamento, equipe, legislação, sistemas legados

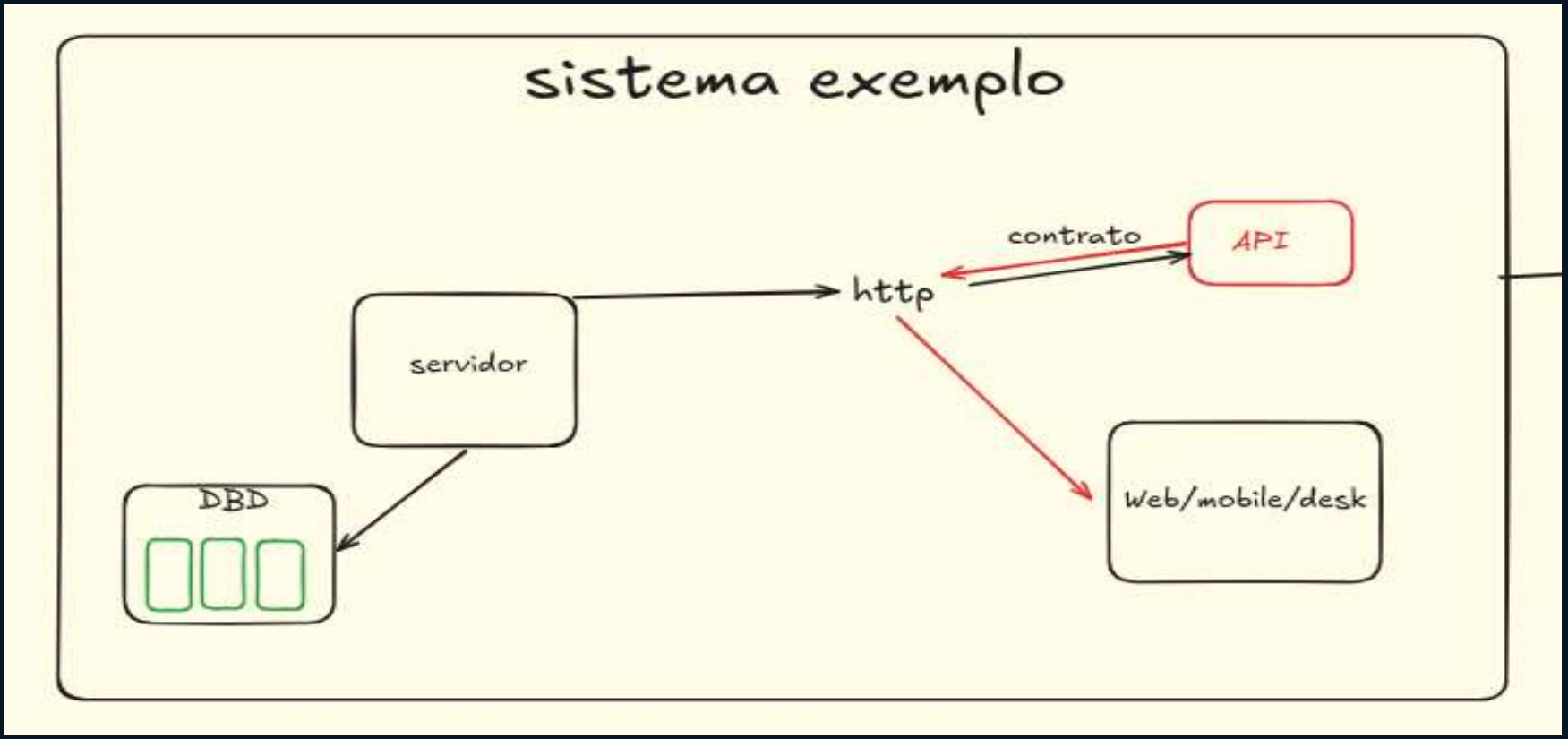
3. Delimite o MVP

- Liste o indispensável para resolver a dor principal.
- Separe “precisa agora” de “pode vir depois”.
- Escreva o que está fora do escopo.
- Evite construir funcionalidades sem validar valor.

4. Desenhe a jornada do usuário

- Exemplo: escolher produto → criar pedido → pagar → acompanhar status.
- Identifique decisões, exceções e caminhos alternativos.
- O que acontece se o pagamento falhar? E se a rede cair?
- Fluxos bem descritos viram casos de teste.

5. Comece com uma arquitetura simples



Desenho inicial: componentes suficientes para o problema atual.

6. Decisões técnicas têm trade-offs

Banco de dados

Consistência, consultas, custo e operação

API e integração

Contrato, autenticação, erros e idempotência

Síncrono ou assíncrono

Resposta imediata versus desacoplamento

Monólito ou serviços

Simplicidade versus autonomia e complexidade

Hospedagem

Escala, manutenção e custo

Observabilidade

Logs, métricas e rastreamento

7. Planeje a implementação

- Divida funcionalidades em entregas pequenas e verificáveis.
- Defina critérios de aceite antes de desenvolver.
- Mapeie dependências: banco, endpoints, telas e integrações.
- Versione decisões importantes e registre riscos.

8. Como o planejamento guia o código

Requisito

Define o comportamento esperado

Fluxo

Define estados, exceções e regras

Modelo de dados

Define entidades e relações

Contrato de API

Define entradas, saídas e erros

Critério de aceite

Define quando está pronto

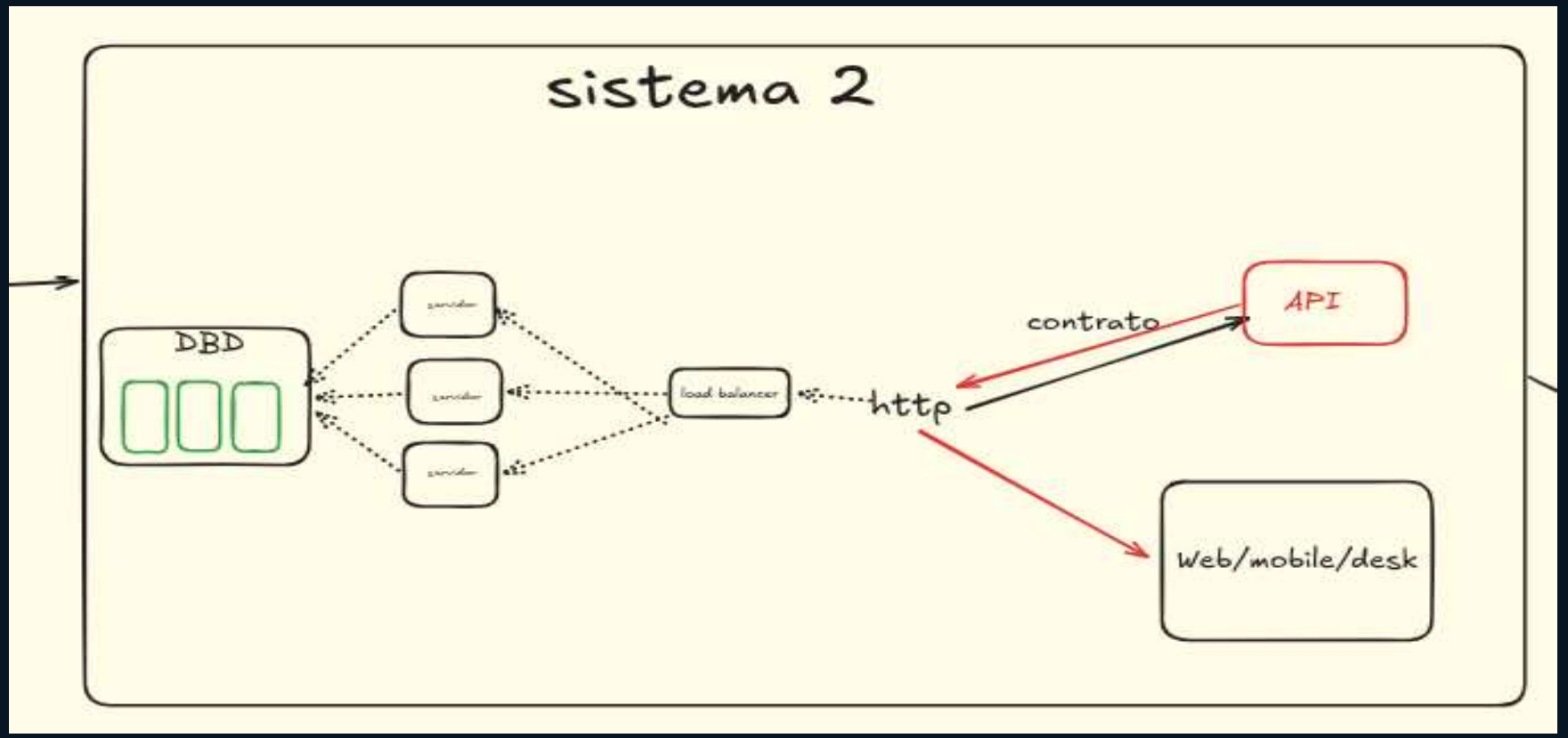
Teste

Comprova o comportamento

9. Testar não é a última etapa

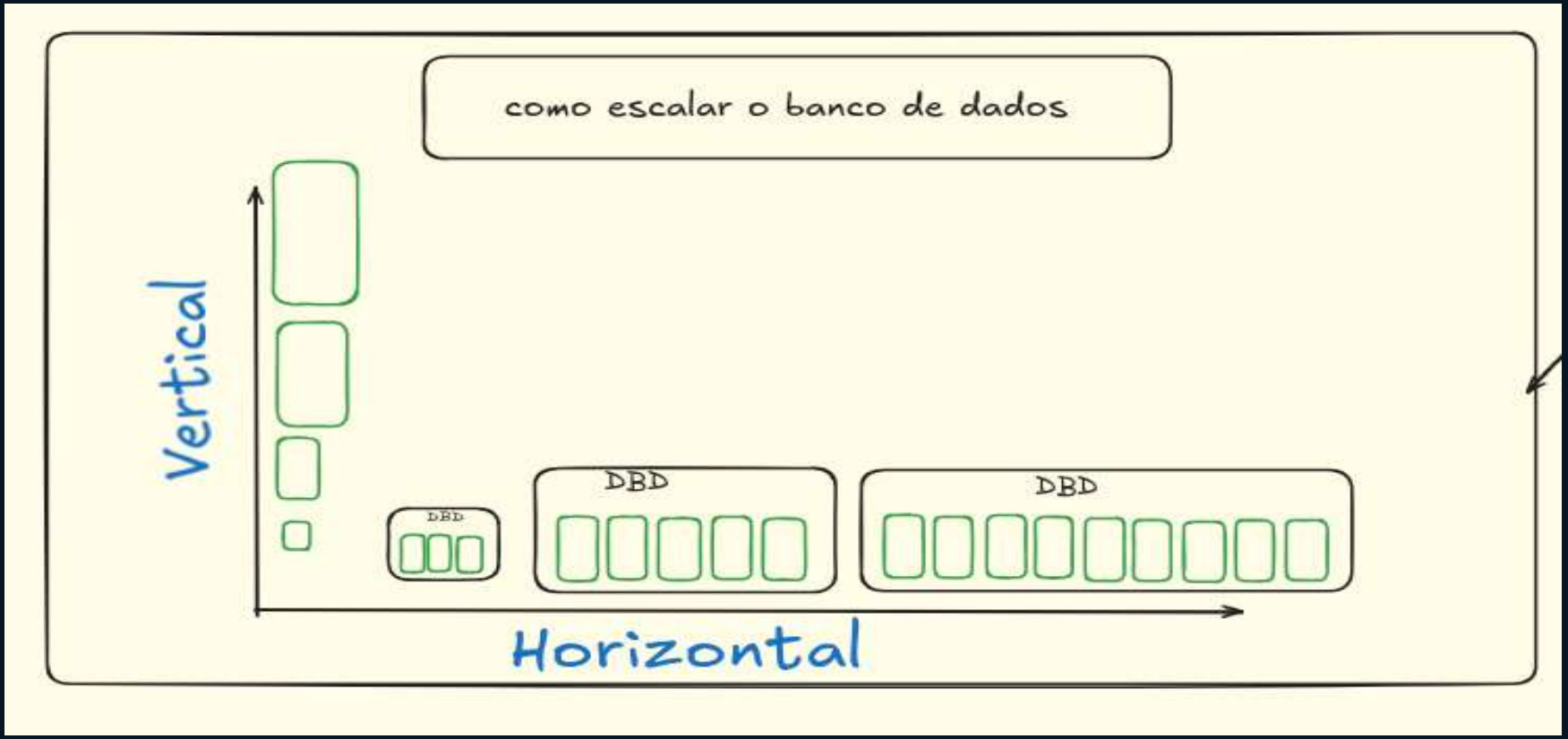
- Teste de unidade: regras e comportamentos isolados.
- Integração: comunicação entre componentes.
- Ponta a ponta: jornada completa do usuário.
- Segurança, falhas e desempenho conforme o risco.

10. O sistema cresce — e agora?



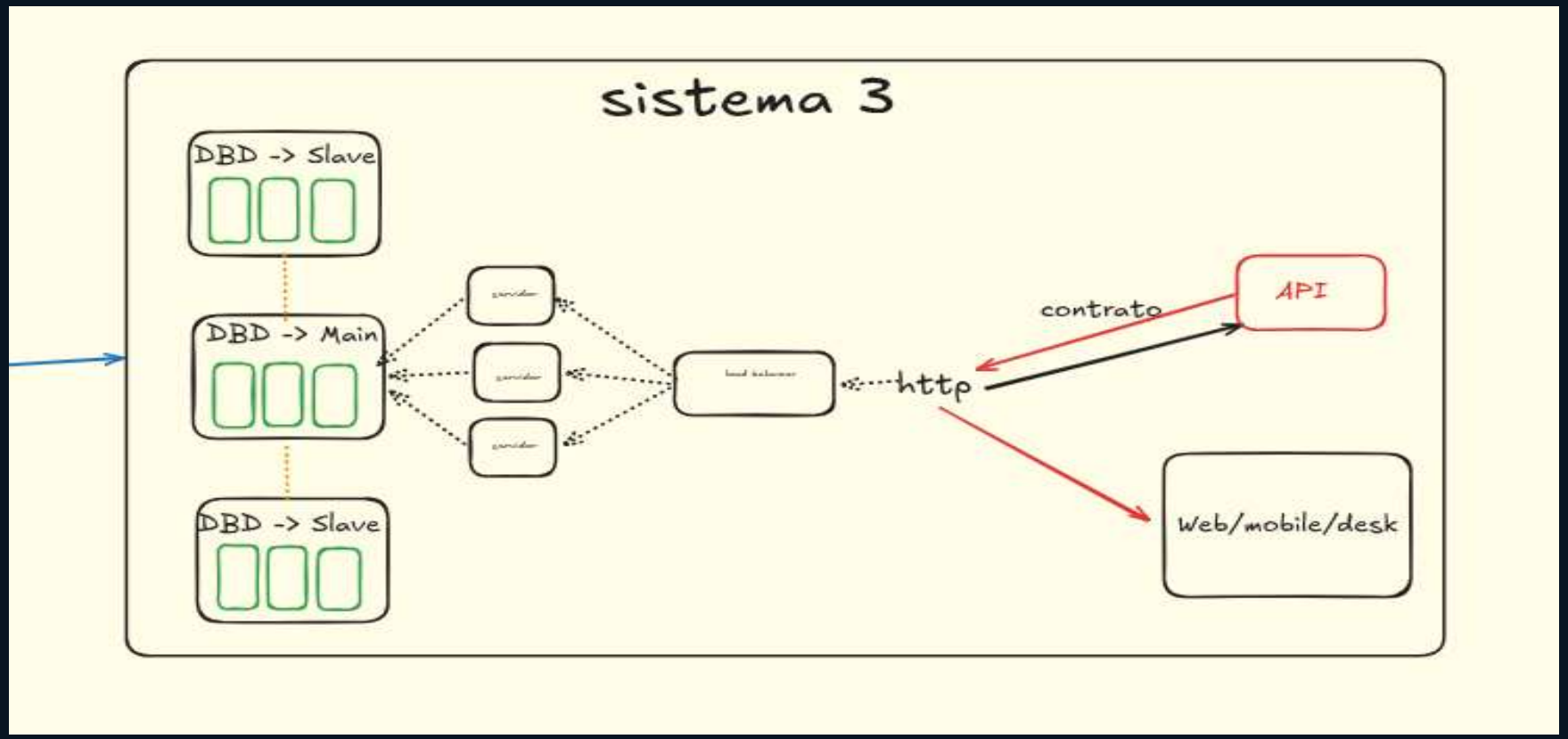
Não escale por moda: observe métricas e identifique o gargalo.

11. Evolua a persistência com evidências



Cada nova camada adiciona benefícios, custos e novos modos de falha.

12. Resiliência e disponibilidade



Replicação exige pensar em consistência, failover e operação.

13. Ciclo de decisão em produção

- Sintoma → medir → formular hipótese → testar → decidir.
- Exemplo: lentidão não significa automaticamente “precisa de microsserviços”.
- Observe latência, erros, volume e uso de recursos.
- Faça a menor mudança que resolva o problema comprovado.

Atividade prática · 15 minutos

Cenário

Uma cantina precisa receber pedidos antecipados

Descoberta

Quem usa? Qual dor? Qual objetivo?

Requisitos

3 funcionais + 2 não funcionais

Fluxo

Desenhe pedido, pagamento e falhas

Arquitetura

Desenhe cliente, API, aplicação e dados

Entrega

Defina MVP e 3 critérios de aceite

Discussão e fechamento

Qual decisão você tomaria antes de escolher a linguagem?

O que você precisaria descobrir para justificar essa escolha?

Lembre-se: código implementa decisões; boas decisões começam com contexto.